《猫语心声》AI 智能 NPC 技术方案设计书

目录

1. 方案概述 2
2. 核心设计理念与评审契合点 2
3. 整体系统架构 2
4. 猫咪个体控制详细设计 3
    4.1 意图决策控制（核心模块） 3
        4.1.1 状态空间定义（MDP） 3
        4.1.2 策略网络（Transformer） 4
        4.1.3 意图集合（枚举） 4
        4.1.4 LLM 职责界定 4
    4.2 动作控制：枚举意图 + 动画状态机 4
    4.3 维护记忆：固定窗口 + 优先级队列 5
        4.3.1 工作记忆（固定窗口） 5
        4.3.2 长期记忆（优先级队列） 5
    4.4 维护人设：性格参数过滤器 5
        4.4.1 意图决策过滤 5
        4.4.2 动画与行为参数过滤 5
        4.4.3 文本生成过滤 5
5. 数据流与交互示例 6
6. 解决“记忆膨胀”与“一致性”的机制总结 6
7. 4 个月 Demo 落地计划（结合本方案） 6
8. 技术风险与缓解 7
9. 结语 7
附录——猫咪个体控制伪代码 8

1. 方案概述

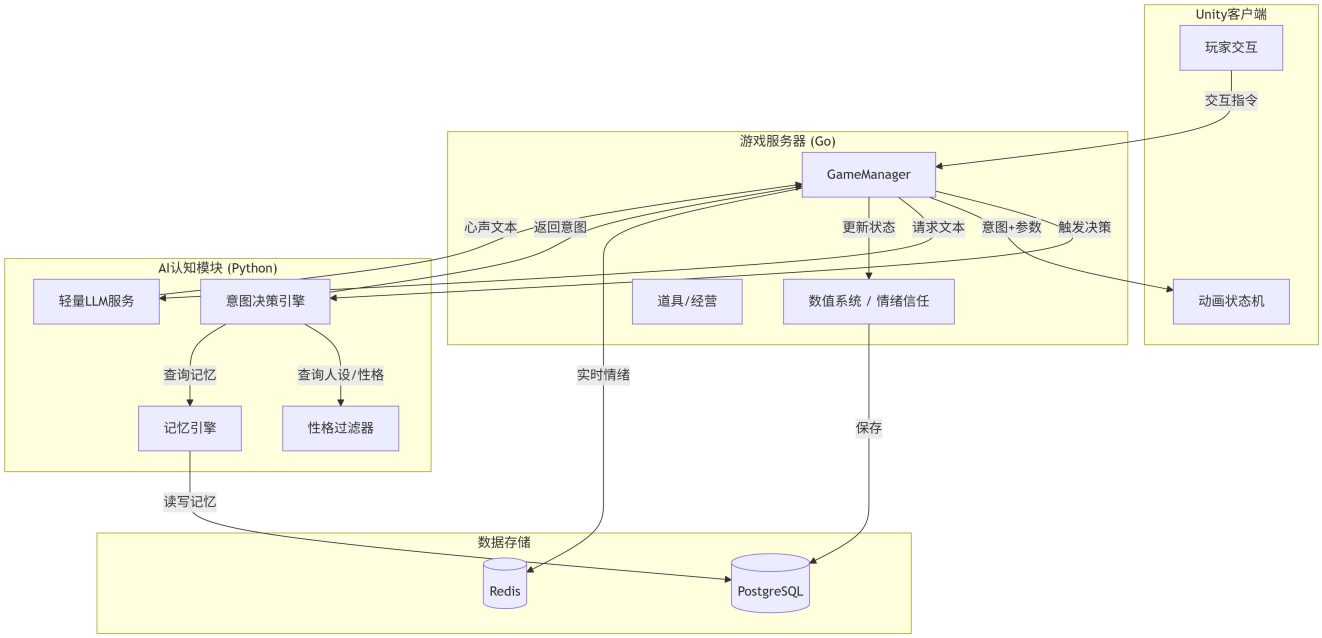
本方案旨在构建一个低成本、低延迟、高一致性的 AI 猫咪系统。核心思路是：将意图决策与文本表达解耦。
- **意图决策**：使用基于 MDP（马尔可夫决策过程）+Embedding+Transformer 的数值模型进行行为选择。
- **文本表达**：仅调用轻量 LLM，依据已决定的意图和相关记忆生成心声文本。
- **记忆与人设**：通过固定窗口+优先级队列进行记忆管理，通过性格参数过滤器保证人设不偏移。

该架构从根源上解决了纯 LLM 驱动 NPC 的三个致命问题：
1. **长线人设漂移**：决策不经过 LLM，避免了语言模型因上下文变化而导致的性格突变。
2. **响应延迟与成本**：高频决策完全本地化，LLM 仅用于相对低频的文本生成，可大幅降低 API 成本与等待时间。
3. **记忆膨胀幻觉**：固定窗口限制工作记忆长度，优先级队列自动淘汰低价值记忆，避免检索噪音。

2. 核心设计理念与评审契合点

评审关注点	本方案应对策略
长线人设一致性	性格参数过滤器（在意图选择、动画表现、文本生成三个环节施加硬约束）+ 持久化的性格向量。
记忆膨胀与幻觉	固定窗口（工作记忆）+ 优先级队列（长期记忆），高价值记忆保留，低价值记忆衰退，无无限文本堆积。
长期情感追求	意图决策中的信任/情绪状态作为 MDP 状态的一部分，随互动累计；信任阈值驱动里程碑事件，形成阶段式追求。
指令跟随与涌现	MDP 策略中融入玩家指令奖励信号；猫咪间社交行为由规则+随机涌现触发，Transformer 评估意图可行性。

3. 整体系统架构




```mermaid
graph TD
    subgraph Client ["Unity 客户端"]
        UI[玩家交互]
```

```
        ASM[动画状态机]
    end

    subgraph Server ["游戏服务器 (Go)"]
        GM[GameManager]
        Num[数值系统 / 情绪信任]
        Inv[道具/经营]
    end

    subgraph AI ["AI 认知模块 (Python)"]
        IDE[意图决策引擎]
        ME[记忆引擎]
        PF[性格过滤器]
        LLM[轻量 LLM 服务]
    end

    subgraph Data ["数据存储"]
        PG[(PostgreSQL)]
        Cache[(Redis)]
    end

    UI -->|交互指令| GM
    GM -->|触发决策| IDE
    IDE -->|查询人设/性格| PF
    IDE -->|查询记忆| ME
    IDE -->|返回意图| GM
    GM -->|意图+参数| ASM
    GM -->|请求文本| LLM
    LLM -->|心声文本| GM
    GM -->|更新状态| Num
    Num -->|保存| PG
    ME -->|读写记忆| PG
    GM -->|实时情绪| Cache
```

4. 猫咪个体控制详细设计

4.1 意图决策控制（核心模块）

意图决策引擎的任务是：根据猫咪当前的状态、环境、玩家行为，输出一个高层意图标签。该意图不包含具体文本，仅如：`friendly_approach`、`fearful_retreat`、`playful_pounce`、`demand_food`。

4.1.1 状态空间定义（MDP）

将每一时刻的猫咪状态定义为一个向量，作为 MDP 的输入状态。向量由以下 Embedding 拼接而成：

```
State = Concat[
```

```
    性格向量(8 维),
    情绪向量(5 维)：{hunger, fear, curiosity, comfort, social}，
    信任度(标量)，
    当前意图历史(one-hot)，
    环境特征向量(区域舒适度、刺激度……)，
    关系向量(对玩家、对其他猫)，
    玩家行为编码(one-hot 或 embedding)
]
```

维度控制在 256 以内，便于 Transformer 处理。

4.1.2 策略网络（Transformer）

采用一个小型 Transformer 编码器（2-4 层，多头注意力），输入为状态序列（当前时刻+前 3 步状态），输出**各个可能意图的 Q 值**或动作概率分布。

- 训练方式：先通过行为克隆（从策划配置的规则行为中采样），再通过在线强化学习（玩家反馈作为奖励信号）微调。
- 决策频率：每秒 1 次（可调），异步执行，不阻塞渲染。
- 优势：决策完全可解释、可干预，不受 LLM 随机性影响。

4.1.3 意图集合（枚举）

所有可能的猫咪意图预先定义为枚举值（约 30-50 个），覆盖所有日常、交互、社交、应激行为。例如：

意图 ID	意图名称	触发条件举例	动作表现
1	idle_wander	无外部刺激	缓慢走动
2	friendly_approach	信任>50 & 玩家靠近	靠近玩家，蹭腿
3	fearful_hide	恐惧>70 & 陌生人进店	逃向隐蔽点
4	play_with_toy	刺激度>30 & 玩具在场	扑玩具
5	social_groom	与其他猫亲密度>60	互相舔毛

4.1.4 LLM 职责界定

LLM 仅用于生成**心声文本**和**夜间社交对话文本**。它接收的数据包包括：

- 猫咪人设文本（固定前缀）
- 当前意图标签及情绪状态
- 从优先级队列中提取的关键记忆片段
- 玩家最近的交互描述

LLM 不参与意图决策，只负责将已定的意图用符合性格的语言表达出来。例如，意图是`fearful_hide`，LLM 生成：“那個腳步聲……好近……我得躲起來……”

4.2 动作控制：枚举意图 + 动画状态机

客户端动画状态机与枚举意图直接映射。服务器下发意图 ID 后，客户端播放对应动画，同时处理过渡。

- 为每个意图设计一个动画状态节点，并设置过渡条件（如`IntentChanged`触发器）。
- 某些意图带参数（如移动到某坐标），通过导航网格+位移实现。

- 心跳气泡的弹出时机与动画事件同步，在特定动画帧触发显示话语。

这种设计保证了动画表现与决策完全同步，且无需实时 AI 干预，降低了编写和维护成本。

4.3 维护记忆：固定窗口 + 优先级队列

记忆系统用于支撑决策和文本生成，但必须避免“记忆膨胀”。采用两级结构：

4.3.1 工作记忆（固定窗口）

- 结构：固定长度的环形队列，最多保留最近 20 条交互记录。
- 内容：摘要描述（如“玩家轻声安抚我”，“被突然的响声惊吓”）+ 时间戳。
- 作用：直接作为 Transformer 决策的输入噪声背景，和 LLM 文本生成的即时上下文。
- 当新记录加入，最旧的记录自然淘汰。

4.3.2 长期记忆（优先级队列）

- 存储：所有记忆都以“重要性评分”排序，仅保留重要性最高的一定数量（如 500 条）。
- 重要性算法：`score = base_importance + time_decay_weight  (1 - (t_now - t_event) / max_ttl)`
  其中`base_importance`由事件类型决定（如“救助里程碑”>“日常喂食”）。
- 当队列满时，分数最低的记忆被移除，实现遗忘。
- 决策/文本生成时，根据语义匹配从长期记忆中提取相关片段（使用本地 Embedding 相似度）。
- 这种设计确保永远不会出现无限记忆导致检索缓慢或 LLM 上下文溢出。

4.4 维护人设：性格参数过滤器

性格参数过滤器是整个系统人设一致性的最后一道关卡。它表现为一个**规则引擎**，在三个环节施加影响：

4.4.1 意图决策过滤

在 Transformer 输出意图概率分布后，过滤器根据性格向量对不合理意图进行惩罚或禁止。
- 例如，对于“小雪”的性格`{怯懦:0.9, 好奇:0.1}`，对意图`friendly_approach`施加大幅负权重，使得该意图几乎不可被选择。
- 实现：在 softmax 之前，给 logits 加上一个与性格兼容性成负相关的偏置。

4.4.2 动画与行为参数过滤

某些动画的播放速度、幅度也受性格影响。例如傲娇猫在“友好蹭腿”动画中会先犹豫再行动，可通过动画层权重控制。
- 扩展行为参数：如“亲近距离阈值”、“恐惧逃跑速度”等，为每只猫存储一份性格系数表。

4.4.3 文本生成过滤

在 LLM 生成文本后，通过一个后处理过滤层检查是否符合性格。
- 使用关键词禁止列表（如对高冷猫禁止生成“好开心”“最喜欢你了”这类语言）。
- 或以轻量分类器（基于性格向量与生成文本的 CLS Embedding 相似度）判断文本是否“人设偏离”，若偏离则要求重新生成或使用备用模板。
- 同时，LLM 的系统指令中已经嵌入了严格的语气要求，双重保证。

5. 数据流与交互示例

以一次玩家“在猫耳视界下轻声安抚流浪猫小雪”为例：

1. 玩家输入通过 Unity 传送至服务器。
2. 服务器提取当前状态向量（情绪+信任+区域等）。
3. 意图决策引擎（Transformer）结合当前状态、工作记忆、优先级记忆中的关键词，预测意图：`fearful_hesitate`（恐惧犹豫）。
4. 性格过滤器对该意图放行（符合怯懦性格）。
5. 服务器下发意图 ID 至客户端，触发“侧头观察，身体微缩”的动画。
6. 同时，服务器调用 LLM，传入格式：
   ```
   [系统] 你是小雪，一只曾受虐待的波斯猫，极度怯懦。当前恐惧值 70，好奇心 10。
   [记忆] 之前玩家轻轻放下食物就离开了；曾经被前主人拖拽……
   [意图] fearful_hesitate
   请用第一人称生成心声。
   ```
7. LLM 返回心声：“这个人类又来了……他没有动，只是蹲在那里……是打算给我吃的吗？还是……”
8. 服务器将心声文本返回客户端，在动画的特定时间点显示气泡。
9. 情绪数值系统根据意图类型更新：恐惧-5，好奇+3。
10. 本次交互作为一条记忆存入工作记忆，重要度评分 6。

整个过程，决策是确定性的（可复现），文本是生成性的（富有变化），两者互不干扰。

6. 解决“记忆膨胀”与“一致性”的机制总结

问题	传统 LLM 方案	本方案
记忆膨胀	将所有历史对话塞入上下文，导致超长 prompt、幻觉和高成本。	固定窗口（20 条）+优先级队列（限制总量），长期记忆只提供相关片段，上下文始终可控。
人设漂移	LLM 容易受对话诱导，逐渐偏离初始人设。	意图层面完全由数值模型+过滤器决定，不随文本变化；文本又受过滤器和 system prompt 双重约束，根源切断人设偏移。
响应速度	每次决策调用大模型，延迟高。	高频意图预测本地完成（<10ms），低频文本生成异步进行，玩家无等待感。
长线情感	信任只是 prompt 中的数字，LLM 难以持续递增。	信任是 MDP 状态的一部分，阈值直接解锁新意图集合和记忆里程碑，情感曲线由数值系统精确控制，文本只是点缀。

7. 4 个月 Demo 落地计划（结合本方案）

该计划与前文一致，但针对本技术方案强化几点：

- **第 1-5 周**：实现意图决策引擎素体，用规则模拟代替 Transformer（快速验证），接入动画状态机和记忆队列，跑通“行为选择→动画→心声生成”闭环。
- **第 6-10 周**：训练一个小型 Transformer 策略网络，替换规则；完善优先级记忆队列与语义检索；加入性格过滤器。
- **第 11-14 周**：完整的流浪猫救助线和情感里程碑；Demo 演示中将重点展示：同一玩法下，小雪与奥利奥因性格不同而产生的不同意图和心声。
- **第 15-16 周**：性能压测，确保决策延迟<30ms，记忆检索<50ms；准备技术答辩材料。

8. 技术风险与缓解

风险	缓解措施
Transformer 策略网络训练数据不足	先用规则（行为树）运行整个 Demo，收集玩家交互数据后离线训练，评委展示时可用混合策略。
LLM 生成的文本与意图情绪不符（如意图是恐惧，但文本很欢快）	在 prompt 中严格绑定意图，后处理用情感分类器做一致性检查，失败则使用策划撰写的备选文本库。
记忆优先级队列的淘汰算法不可靠	提供 GM 调试界面，可手动锁定重要记忆；线上初期使用保守阈值，后期调整。
动画状态机复杂度膨胀	意图枚举控制在 30 个以内，许多细微差异通过动画层混合实现，而非独立状态。

9. 结语

本方案通过**意图决策与文本表达的彻底解耦**，构建了一个稳固、高效、可信赖的 AI 猫咪系统。在智能 NPC 赛道上，它不仅满足了“长期记忆、性格定制、深度互动”的要求，更以一种**非纯黑箱 LLM** 的方式，展现了游戏 AI 工程化的可行路径。我们有信心在 4 个月内打造出一个让玩家与评委都感受到温暖的 Demo。

附录——猫咪个体控制伪代码

```
import numpy as np
import random
from collections import deque
import heapq
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import List, Tuple, Optional, Dict

# ==================== 配置常量 ====================
INTENT_LIST = [
    "idle_wander", "friendly_approach", "fearful_retreat",
    "playful_pounce", "demand_food", "social_groom",
    "curious_inspect", "hide", "sleep", "scratch_furniture",
    "accept_petting", "hiss_warning", "investigate_sound",
    "follow_player", "stare_at_window"
]  # 枚举全部意图，约 30 个，此处简化为 15 个示例

EMBED_DIM = 64              # 各子特征嵌入维度
STATE_DIM = 256             # 拼接后状态向量维度
SEQ_LEN = 4                 # 输入 Transformer 的序列长度（当前 + 前 3 步）
INTENT_NUM = len(INTENT_LIST)
WORK_MEMORY_SIZE = 20    # 固定窗口工作记忆大小
LONG_MEMORY_CAP = 500    # 长期记忆最大容量

# 性格维度（示例 8 维）
PERSONALITY_KEYS = ["傲娇", "怯懦", "好奇", "贪吃", "社交", "攻击性", "活跃", "独立"]

# ==================== 数据类定义 ====================
class MemoryItem:
    """记忆单元（用于工作记忆和长期记忆）"""
    def __init__(self, desc: str, timestamp: float, importance: float, embedding: Optional[np.ndarray] = None):
        self.desc = desc                # 文本描述（如“玩家轻声安抚我”）
        self.timestamp = timestamp
        self.importance = importance   # 0.0~10.0，越高越重要
        self.embedding = embedding      # 预计算的语义向量（可选，用于检索）

    # 用于优先队列比较（大顶堆）
    def __lt__(self, other):
        return self.importance > other.importance

class CatState:
    """猫咪当前状态（MDP 状态的一部分）"""
    def __init__(self):
        self.personality_vector = np.zeros(len(PERSONALITY_KEYS))   # 8 维，值域[0,1]
        self.emotion_vector = np.zeros(5)     # [饥饿, 恐惧, 好奇, 舒适, 社交]
```

```
        self.trust_level = 0.0                    # 信任度 0~100
        self.energy = 1.0                         # 精力 0~1
        self.health = 1.0
        self.current_room_id = 0
        self.relationship_to_player = 0.0     # 对玩家的好感
        # 环境特征（由房间舒适度/刺激度等决定，此处简化）
        self.environment_features = np.zeros(5)

    def to_embedding(self) -> np.ndarray:
        """将状态拼接为初始 embedding 向量（各子特征分别 embedding 后拼接）"""
        # 实际实现中会用 Embedding 层将各离散/连续特征映射到 EMBED_DIM 维再拼接
        # 这里简化为归一化后的直接拼接，但保留接口
        return np.concatenate([
            self.personality_vector,
            self.emotion_vector,
            [self.trust_level / 100.0],
            [self.energy],
            [self.health],
            self.environment_features,
            [self.relationship_to_player / 100.0]
        ])

# ==================== 记忆管理器 ====================
class MemoryManager:
    def __init__(self):
        self.working_memory = deque(maxlen=WORK_MEMORY_SIZE)   # 固定窗口
        self.long_term_memory = []   # 最小堆，容量限制
        self.semantic_index = None   # 可选的向量索引（用 numpy 或 faiss 等）

    def add_memory(self, item: MemoryItem):
        """添加一条记忆到工作记忆，并决定是否进入长期记忆"""
        self.working_memory.append(item)
        # 仅当重要性超过阈值或特殊类型事件才存入长期记忆
        if item.importance > 4.0:
            heapq.heappush(self.long_term_memory, item)
            # 保持长期记忆容量
            while len(self.long_term_memory) > LONG_MEMORY_CAP:
                heapq.heappop(self.long_term_memory)   # 弹出最低重要性的

    def get_recent_memories(self, k: int = 5) -> List[MemoryItem]:
        """获取最近 k 条工作记忆，用于决策"""
        return list(self.working_memory)[-k:]

    def query_long_term(self, query_embedding: np.ndarray, top_k: int = 3) -> List[MemoryItem]:
        """根据语义向量检索相关长期记忆（简化：用余弦相似度）"""
        if not self.long_term_memory:
            return []
        # 假设每条长期记忆都有 embedding，这里模拟
        scored = []
```

```
        for mem in self.long_term_memory:
            if mem.embedding is not None:
                sim = np.dot(query_embedding, mem.embedding)
                scored.append((sim, mem))
        scored.sort(key=lambda x: x[0], reverse=True)
        return [mem for sim, mem in scored[:top_k]]

# ==================== 性格过滤器 ====================
class PersonalityFilter:
    """根据性格向量对意图 logits 进行过滤和修正"""
    def __init__(self, personality_config: Dict[str, Dict[str, float]]):
        """
        personality_config: 定义每种性格对不同意图的偏好权重。
        例如：{'怯懦': {'friendly_approach': -5.0, 'hide': 3.0}, ...}
        """
        self.config = personality_config

    def apply(self, logits: torch.Tensor, personality_vec: np.ndarray, intent_names: List[str]) -> torch.Tensor:
        """
        logits: [batch, num_intents]
        personality_vec: [batch, num_personality_traits]
        返回修正后的 logits
        """
        # 根据性格向量生成每个意图的偏置
        batch_size = logits.shape[0]
        bias = torch.zeros_like(logits)
        for i, intent in enumerate(intent_names):
            # 对每个性格维度，如果存在配置，就加和
            for j, trait in enumerate(PERSONALITY_KEYS):
                weight = self.config.get(trait, {}).get(intent, 0.0)
                bias[:, i] += weight * personality_vec[:, j]
        return logits + bias

    def filter_text(self, generated_text: str, personality_vec: np.ndarray, cat_name: str) -> str:
        """
        文本后处理过滤：检查是否违背人设。
        简化实现：关键词屏蔽或情感极性检查。
        如果检查失败，返回备用的中性表达。
        """
        # 示例：怯懦高的猫不应说出太自信的话
        if personality_vec[PERSONALITY_KEYS.index("怯懦")] > 0.7:
            forbidden = ["最喜欢", "主动过来", "我不怕"]
            for word in forbidden:
                if word in generated_text:
                    return f"（{cat_name}犹豫了一下，没有开口）"
        return generated_text

# ==================== 基于 Transformer 的意图策略网络 ====================
class IntentTransformerPolicy(nn.Module):
```

```
    def __init__(self, state_dim=STATE_DIM, num_intents=INTENT_NUM, seq_len=SEQ_LEN):
        super().__init__()
        self.seq_len = seq_len
        self.state_proj = nn.Linear(state_dim, EMBED_DIM)  # 将状态映射到 embed_dim
        encoder_layer = nn.TransformerEncoderLayer(d_model=EMBED_DIM, nhead=4, dim_feedforward=128,
batch_first=True)
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=2)
        self.out_head = nn.Linear(EMBED_DIM, num_intents)

    def forward(self, state_sequence: torch.Tensor) -> torch.Tensor:
        """
        state_sequence: [batch, seq_len, state_dim]
        返回意图 logits [batch, num_intents]
        """
        # 投影
        x = self.state_proj(state_sequence)  # [batch, seq_len, embed_dim]
        # Transformer 编码
        x = self.encoder(x)  # [batch, seq_len, embed_dim]
        # 取最后一个时间步的特征
        x_last = x[:, -1, :]
        logits = self.out_head(x_last)
        return logits

# ==================== 轻量 LLM 服务（模拟接口） ====================
class LightweightLLM:
    def generate_inner_monologue(self, prompt: str) -> str:
        """
        调用轻量 LLM 生成猫咪心声文本。
        实际实现：调用本地模型或 API，此处模拟返回。
        """
        # 模拟基于 prompt 的简单回应
        if "fearful" in prompt.lower():
            return "好可怕... 我想躲起来。"
        elif "hungry" in prompt.lower():
            return "肚子有点饿了，那个人类会给我吃的吗？"
        elif "play" in prompt.lower():
            return "那个玩具好像很有意思... 但是不是陷阱？"
        else:
            return "（静静地观察着周围）"

# ==================== 猫咪 Agent（整合所有模块） ====================
class CatAgent:
    def __init__(self, cat_id: str, name: str, personality: np.ndarray,
                 personality_filter_config: dict, intent_model: IntentTransformerPolicy, llm: LightweightLLM):
        self.cat_id = cat_id
        self.name = name
        self.state = CatState()
        self.state.personality_vector = personality
        self.filter = PersonalityFilter(personality_filter_config)
```

```
        self.memory_mgr = MemoryManager()
        self.intent_model = intent_model
        self.llm = llm
        self.intent_history = deque(maxlen=SEQ_LEN)   # 记录最近几个意图（用于状态序列）
        # 初始意图为 idle
        self.current_intent = "idle_wander"

    def process_interaction(self, player_action: str, environment: dict) -> Tuple[str, str]:
        """
        处理一次玩家交互，返回 (意图名称, 心声文本)
        """
        # 1. 更新环境特征（简化）
        self.state.environment_features = self._extract_env_features(environment)

        # 2. 构造当前状态向量（序列形式）
        state_now = self._build_state_vector(player_action)
        state_seq = self._prepare_state_sequence(state_now)

        # 3. 通过 Transformer 获取意图 logits
        with torch.no_grad():
            logits = self.intent_model(state_seq.unsqueeze(0))   # [1, num_intents]

        # 4. 性格过滤器修正 logits
        personality_tensor = torch.tensor([self.state.personality_vector], dtype=torch.float32)
        filtered_logits = self.filter.apply(logits, personality_tensor, INTENT_LIST)

        # 5. 采样或 argmax 得到意图
        intent_idx = torch.argmax(filtered_logits, dim=1).item()
        selected_intent = INTENT_LIST[intent_idx]
        self.current_intent = selected_intent
        self.intent_history.append(selected_intent)

        # 6. 更新情绪/信任（简化：根据意图和玩家行为应用规则）
        self._update_emotional_state(selected_intent, player_action)

        # 7. 记忆存储（本次交互记忆）
        mem = MemoryItem(
            desc=f"玩家{player_action}，我选择了{selected_intent}",
            timestamp=environment.get("time", 0),
            importance=self._evaluate_importance(selected_intent, player_action)
        )
        self.memory_mgr.add_memory(mem)

        # 8. 准备 LLM 提示，生成心声文本
        inner_monologue = self._generate_monologue(player_action, selected_intent)

        return selected_intent, inner_monologue

    def _build_state_vector(self, player_action: str) -> np.ndarray:
```

```
        """将包括玩家行为在内的全部信息转为状态向量"""
        # 玩家行为简单编码（实际可用 embedding，此处 one-hot 模拟）
        action_embed = np.zeros(10)  # 假设 10 种玩家行为
        action_list = ["pet", "feed", "call", "play", "ignore", "scold", "approach", "leave", "treat", "heal"]
        if player_action in action_list:
            action_embed[action_list.index(player_action)] = 1.0

        # 基础状态
        base = self.state.to_embedding()
        return np.concatenate([base, action_embed])

    def _prepare_state_sequence(self, current_state: np.ndarray) -> torch.Tensor:
        """用历史状态填充序列，不足补 0"""
        seq = []
        # 从意图历史重建之前的状态（简化：只用当前状态填充，实际应存储历史状态）
        # 这里为了演示，全部使用当前状态填充
        for _ in range(SEQ_LEN):
            seq.append(current_state)
        return torch.tensor(seq, dtype=torch.float32)

    def _extract_env_features(self, environment: dict) -> np.ndarray:
        """提取环境特征向量"""
        return np.array([
            environment.get("comfort", 0.5),
            environment.get("stimulation", 0.3),
            environment.get("hygiene", 0.8),
            environment.get("light", 0.6),
            environment.get("noise", 0.2)
        ])

    def _update_emotional_state(self, intent: str, player_action: str):
        """根据意图和玩家行为调整情绪和信任（简化规则）"""
        if intent == "fearful_retreat":
            self.state.emotion_vector[1] = min(1.0, self.state.emotion_vector[1] + 0.1)  # 恐惧+
        elif intent == "friendly_approach":
            if player_action in ["pet", "treat"]:
                self.state.trust_level = min(100, self.state.trust_level + 2)
                self.state.emotion_vector[3] += 0.1  # 舒适增加
        # 其他更多规则...

    def _evaluate_importance(self, intent: str, player_action: str) -> float:
        """事件重要性评分（简化）"""
        important_intents = ["fearful_retreat", "friendly_approach", "hiss_warning"]
        if intent in important_intents:
            return random.uniform(6.0, 9.0)
        else:
            return random.uniform(2.0, 5.0)

    def _generate_monologue(self, player_action: str, intent: str) -> str:
```

```
        """调用 LLM 生成心声文本，注入记忆和性格约束"""
        # 检索相关记忆
        query_emb = self.state.to_embedding()  # 用当前状态作查询
        mems = self.memory_mgr.query_long_term(query_emb, top_k=3)
        memory_context = "; ".join([m.desc for m in mems])

        # 构建 prompt
        personality_desc = ", ".join([f"{k}:{v:.2f}" for k, v in zip(PERSONALITY_KEYS, self.state.personality_vector)])
        prompt = f"""你是一只名叫{self.name}的猫咪。性格: {personality_desc}。
当前情绪: 饥饿 {self.state.emotion_vector[0]:.2f}, 恐惧 {self.state.emotion_vector[1]:.2f}, 好奇 {self.state.emotion_vector[2]:.2f}。
信任度: {self.state.trust_level:.1f}。
刚刚玩家行为: {player_action}。你选择的意图: {intent}。
相关记忆: {memory_context}
请用第一人称的内心独白表达你的感受和想法，保持性格一致性："""

        raw_text = self.llm.generate_inner_monologue(prompt)
        # 性格文本过滤
        filtered_text = self.filter.filter_text(raw_text, self.state.personality_vector, self.name)
        return filtered_text

# ==================== 示例运行 ====================
if __name__ == "__main__":
    # 初始化性格配置
    personality_config = {
        "怯懦": {"friendly_approach": -5.0, "hide": 3.0, "curious_inspect": -2.0},
        "好奇": {"curious_inspect": 3.0, "investigate_sound": 2.5},
        "傲娇": {"accept_petting": -2.0, "follow_player": -1.0},
        # 其余性格省略...
    }

    # 实例化模型（简化：随机初始化）
    intent_model = IntentTransformerPolicy()
    llm = LightweightLLM()

    # 创建猫咪“小雪”，性格偏怯懦、好奇低
    xiaoxue_personality = np.array([0.0, 0.9, 0.1, 0.2, 0.1, 0.0, 0.1, 0.3])  # 对应 8 维
    cat = CatAgent("cat_001", "小雪", xiaoxue_personality, personality_config, intent_model, llm)

    # 模拟一次玩家互动
    intent, voice = cat.process_interaction("approach", {"time": 1000, "comfort": 0.7, "stimulation": 0.2})
    print(f"意图: {intent}")
    print(f"心声: {voice}")

    # 再喂食
    intent, voice = cat.process_interaction("treat", {"time": 1001, "comfort": 0.7, "stimulation": 0.2})
    print(f"意图: {intent}")
    print(f"心声: {voice}")
```